

Lava Completeness Program — Master Report & Phased Implementation Plan

For agentic workers: REQUIRED SUB-SKILL: Use superpowers:subagent-driven-development (recommended) or superpowers:executing-plans to implement each phase. Large phases (4–9) spawn their own detailed sub-plan under docs/superpowers/plans/ before execution begins — this master plan defines *what* and *acceptance criteria*; each sub-plan defines *every bite-sized TDD step with full code*. Steps use checkbox (- []) syntax.

Goal: Drive every Lava module/app/library/test/doc to a finished, connected, fully-documented, 100%-behaviorally-covered, leak/deadlock/race-free, responsiveness-optimized state — with no dead code, no disabled features, and no undocumented surface — while never breaking existing working behavior and never starting an interactive (root/sudo) process.

Architecture: Phased program. Phase 0 builds the safety net + ground-truth baselines. Phases 1–3 close concrete code gaps (dead/unfinished code, scanning infra, concurrency hardening). Phases 4–5 push tests to maximum (all types + stress/load/chaos + metrics-driven optimization). Phases 6–9 complete documentation/manuals/courses/website/diagrams/SQL. Phase 10 drains constitutional debt. Phase 11 verifies + distributes. Every change is TDD-first, falsifiability-rehearsed (\$6.J), and gated by the existing scripts/ci.sh + pre-push apparatus.

Tech Stack: Kotlin/Compose + Orbit MVI (:app, feature/*, core/*); Go/Gin/HTTP3 (lava-api-go); Kotlin JNI on-device API (:api-app, :core:apiengine); Room (SQLite) + Postgres migrations; Podman/Docker via submodules/containers; Prometheus/Loki/Tempo/Grafana; JaCoCo/Kover (Kotlin coverage), go test -cover (Go coverage); Snyk + SonarQube (to be added, containerized); JUnit4 + orbit-test + Compose UI Challenge Tests + Go contract/parity/e2e tests.

Honest Constraints (read before estimating)

1. **Device/Challenge gates cannot execute on this macOS host.**
§6.AH-debt is OPEN: the podman VM on darwin/arm64 exposes no /dev/kvm/HVF passthrough to a container, and §6.AH forbids host-direct emulators. Every step that runs Compose UI Challenge Tests, connectedAndroidTest, the §6.Z pre-distribute gate, or the §6.AE matrix **MUST run on a Linux x86_64+KVM gate-host** (or the real operator-provided S23 Ultra for spot verification). Steps are written to be host-portable; they are marked **[GATE-HOST]**.
 2. **No interactive/root processes.** Snyk + SonarQube run as rootless-Podman containers via Compose; no sudo, no su, no host package installs requiring elevation (§6.U). Any tool that *only* installs via sudo is rejected and containerized instead.
 3. **"100% coverage" = behavioral maximum, not lexical theatre.**
Per §6.D coverage is measured behaviorally; literal-line 100% is the target where it is *meaningful*, and every intentionally-uncovered line goes in a per-module coverage-exemption ledger with a named reason (no blanket waivers). A green coverage number on a bluff test is worse than no test (§6.J).
 4. **No-break guarantee.** Every phase starts from a captured green baseline (Phase 0) and every task is additive-or-refactor-with-passing-tests. Any change that reddens a pre-existing green test is reverted before commit.
 5. **Multi-mirror + bump discipline.** §6.W (GitHub+GitLab only), §6.Y (version bump after each distribute), §6.S (CONTINUATION.md updated in the same commit), §6.P (CHANGELOG per distribute) apply throughout.
-

PART I — COMPLETENESS REPORT (Current Ground-Truth State)

A. Artifact status

Artifact	Path	State	Notes
:app Android client	app/	ACTIVE / shipping	1.3.1-1058 (post-distribute bump); 1.3.0-1057 distributed 2026-06-04
lava- api-go Go API	lava-api- go/	ACTIVE / SP-2 complete	2.3.23-2323; OCI image + binary; HTTP/3 primary
:api-app on- device API	api-app/	ACTIVE / recently shipped	0.2.1-5; JNI bridge to lava-api-go; C01–C04 green
:proxy Ktor server	proxy/ already removed 2026-05-06	GONE	No proxy/ dir, 0 tracked files. build_and_release.sh/docker- compose.yml/start.sh carry "legacy Ktor proxy was removed" comments. The earlier "preserved as fallback via --legacy" claim was stale/ wrong. Operator decision: delete entirely → already satisfied; Phase 1 only verifies- gone + scrubs stale wording.

B. Dead / unfinished / disabled code (verified, with paths)

Genuinely unfinished (must be wired):

- feature/topic/src/main/kotlin/lava/topic/TopicScreen.kt — two
Orbit side-effects unwired: is
TopicSideEffect.ShowAddCommentDialog -> Unit // TODO and is

TopicSideEffect.ShowAddCommentError -> Unit // TODO. **Whole add-comment feature is dead-ended at the UI layer.**

- core/designsystem/.../component/FloatingActionButton.kt (×2) and .../component/Placeholder.kt (×1) — contentDescription = null, // TODO → accessibility gap (TalkBack).

Stubs that are intentional (verify, don't "fix"):

- core/testing/.../Test{Bookmarks,Favorites,Suggests,Visited} Repository.kt — TODO("Not yet implemented") in unused methods. These are test doubles; per §3 (Third Law) they must be **behaviorally equivalent where used**. Audit: confirm no used method throws; implement any that production tests actually call.

Conditional, documented (leave as-is, document):

- lava-api-go/internal/observability/nonfatal.go — Firebase Crashlytics REST bridge gated on LAVA_API_FIREBASE_CRASHLYTICS_ENABLED. Graceful degradation; document, don't change.
- lava-api-go/internal/rutacker/provider.go — // TODO: revisit ... multi-provider auth flow. SP-3a-bridge scope; document the boundary.

Inherited submodule (out of Lava's direct scope, track upstream):

- Submodules/HelixQA/pkg/capture/windows_capture.go — Windows frame-read unimplemented. Non-gating; macOS+Linux paths active.

:proxy — dead in the release pipeline (not Gradle-built, not distributed). Decision artifact required.

C. Test state & coverage gaps

- **Test types PRESENT:** JUnit4 unit, orbit-test ViewModel, Compose UI Challenge Tests (app/src/androidTest/.../challenges/Challenge*Test.kt, ~37 classes), Go contract/parity/e2e/integration (lava-api-go/tests/*), :api-app C01–C04, falsifiability-rehearsal harness.
- **Test types ABSENT or thin:** dedicated **stress** tests, **load** tests, **mutation** testing wired as a gate, **fuzz** tests (Go has native fuzzing — unused), **screenshot/visual-diff** tests, **metrics/monitoring-assertion** tests, **chaos** tests beyond §11.4.85 phase-1.

- **Coverage posture:** no JaCoCo/Kover aggregate coverage report exists; several core/* and feature/* modules have sparse or no src/test. A baseline must be captured (Phase 0) before any "increase to max" claim.
- **Coverage tooling:** not configured (no Kover plugin, no aggregate go test -coverprofile rollup).

D. Documentation / manuals / courses / website / diagrams / SQL

Category	State	Gap
docs/ core	EXISTS-CURRENT	Missing: REST API reference , deployment guide , security hardening , DB ERD
User manuals	PARTIAL	Only docs/guides/ON_DEVICE_API_USER_GUIDE.md; no client app manual, install/setup, feature tutorials, troubleshooting
Video courses	ABSENT (main repo)	Submodules have docs/courses/lesson-*.md (not Lava-app). No Lava client/API course.
Website	ABSENT in-repo	External lava-app.tech only; no website/ / mkdocs / docusaurus in repo
Diagrams	ABSENT in docs/	Submodules have .mmd; main docs have prose only, no diagram assets
SQL defs	CURRENT	Room schemas/5–11.json, Postgres migrations/0001–0009; no unified ERD / schema.md
README/ CHANGELOG/ AGENTS	CURRENT	OK

E. Concurrency & safety posture

- **No critical hazards found.** Kotlin tracker HTTP clients use Semaphore(4) + ConcurrentHashMap + @Synchronized cookie jar + circuit breakers. Go uses sync.RWMutex, buffered error channels,

context + defer cancel(), sync.Once/WaitGroup for GC lifecycle, SQLite WAL + single-writer pool.

- **Available-but-unused reuse:** submodules/concurrency/pkg/{semaphore, bulkhead, lazyloader, pool, limiter, safe, ...} and submodules/ratelimiter/pkg/* are not wired into core modules (inline impls used instead). Opportunity: consolidate onto submodule primitives (Decoupled Reusable Architecture rule) + add lazy-init/non-blocking where audits find blocking spots.
- **Audit still owed:** systematic leak (listener/observer/NsdManager/BroadcastReceiver lifecycle), deadlock (lock-ordering), and race (shared var) sweep with reproducing tests — no such sweep exists today.

F. Security / quality scanning gaps

- **Snyk:** ABSENT. **SonarQube:** ABSENT. **Detekt:** ABSENT. **Semgrep:** not in CI. **go vet/golangci-lint:** not gated.
- **Present:** Spotless+ktlint, check-constitution.sh, 3× scan-no-hardcoded-*.sh, check-non-fatal-coverage.sh, check-challenge-{discrimination, coverage}.sh, codegraph.
- **Containers infra exists** (docker-compose.yml + submodules/containers) → Snyk CLI + SonarQube+Scanner can be added as rootless-Podman Compose services.

G. Open constitutional debt (release-relevant)

§6.Y-debt, §6.Z-debt, §6.AA-debt (partial), §6.AB-debt, §6.AC-debt, §6.AD-debt (partial), §6.AE-debt (advisory), §6.X/§6.AH-debt (device-gate boot on macOS), §11.4.109 UNCONFIRMED: marker, docs_chain submodule incorporation. None block the *current* shipped builds; all are in scope for "nothing unfinished."

H. CONTINUATION.md "OWED" items (carry forward)

Cloud-search auth lag (thinker.local registration of android-1.3.0-1057), C02 on-device rehearsal, §6.AH containerized/TCG emulator boot, §11.4.109 verbatim quote, docs_chain add.

PART II — PHASED IMPLEMENTATION PLAN

Each phase lists: **Objective** · **Files** · **Tasks** · **Test/Challenge coverage** · **Constitution gates** · **Exit criteria**. Phases 4–9 each get a dedicated sub-plan authored at phase start (they are too large for full inline code here without fabricating speculative content — which the writing-plans skill forbids). Phases 0–3 + 10–11 carry concrete bite-sized tasks inline.

Phase 0 — Safety net + ground-truth baselines (*no feature change*)

Objective: Capture an immutable green baseline + coverage/perf/scan baselines so "no regression" and "increased coverage" are provable, not asserted.

Files:

- Create: `.lava-ci-evidence/completeness-program/2026-06-04-baseline/` (build logs, coverage reports, scan outputs, perf numbers)
- Create: `docs/superpowers/specs/2026-06-04-completeness-program-design.md` (the spec this plan derives from — backfill)
- Modify: `docs/CONTINUATION.md` (§6.S — record program start)



Step 1: Hardlink .git backup (§9 absolute-data-safety). Run `cp -al .git .git-backup-pre-completeness-$(date +%Y%m%d-%H%M%S).git.mirror` (non-interactive, no sudo).



Step 2: Capture full build green. `./gradlew assembleDebug :lava-api-go equivalent + cd lava-api-go && make build; tee logs to baseline dir. Expected: BUILD SUCCESSFUL.`



Step 3: Capture unit-test green. `./gradlew testDebugUnitTest --continue 2>&1 | tee .../unit-tests.log; cd lava-api-go && go test ./... 2>&1 | tee .../go-tests.log. Record pass/fail counts.`



Step 4: Capture coverage baseline. (Phase 2 wires the tooling; here, capture whatever exists + record "no aggregate report yet" honestly.)



Step 5: Capture forbidden-files + constitution green. `./scripts/check-constitution.sh; ./scripts/ci.sh --changed-only → tee to baseline.`



Step 6: Commit baseline (`chore(completeness): capture green baseline + program spec`), update `CONTINUATION.md`.

Exit criteria: Baseline dir contains build+test+scan logs with explicit pass/fail counts; spec doc committed; `CONTINUATION.md` notes program start.

Phase 1 — Close verified dead/unfinished code (*TDD, falsifiability-rehearsed*)

Objective: Wire every confirmed unfinished/dead surface; decide :proxy fate; audit intentional stubs.

Task 1.1 — Wire `TopicSideEffect.ShowAddCommentDialog` + `ShowAddCommentError`

Files:

- Modify: `feature/topic/src/main/kotlin/lava/topic/TopicScreen.kt`
- Modify (if needed): `feature/topic/src/main/kotlin/lava/topic/TopicViewModel.kt`, state/side-effect sealed types
- Test: `feature/topic/src/test/kotlin/lava/topic/TopicViewModelTest.kt`
- Challenge: `app/src/androidTest/kotlin/lava/app/challenges/Challenge<N>TopicAddCommentTest.kt`



Step 1: Read `TopicViewModel` + `TopicState/TopicSideEffect` to learn the existing add-comment action surface (is there an `onAddCommentClick`? does the VM post the side-effect?).



Step 2 (test-first): Write `TopicViewModelTest` orbit-test asserting: invoking the add-comment action posts `ShowAddCommentDialog`; a failing submit posts `ShowAddCommentError` with the real error. Use

the **real** UseCase + a fake repo that enforces the production error contract (Second/Third Law).



Step 3: Run → FAIL.



Step 4: Implement the dialog state + handlers in `TopicScreen.kt` (replace both `-> Unit // TODO` with real Compose dialog show/dismiss + error snackbar via `core:designsystem`). Wire submit through the existing comment UseCase if present; if absent, this becomes its own sub-task with the UseCase added.



Step 5: Run unit test → PASS.



Step 6 [GATE-HOST]: Write + run the Challenge driving open-topic → tap add-comment → see dialog → submit → see result.

Falsifiability rehearsal: break the post (return before posting side-effect) → Challenge fails with a clear message → revert.



Step 7: `spotlessApply`; commit with Bluff-Audit stamp (\$6.J/\$7.1).

Task 1.2 — Accessibility `contentDescription` (3 sites)

Files: `core/designsystem/.../component/FloatingActionButton.kt` (×2), `.../Placeholder.kt` (×1); **Test:** `core/designsystem/src/test/.../AllyContentDescriptionTest.kt` (Robolectric/semantics) or a Challenge semantics assertion.



Step 1: Decide API — add a required/overridable `contentDescription`: `String` param (FAB) and a sensible default for `Placeholder`. Match neighboring component signatures.



Step 2 (test-first): Assert the rendered node exposes a non-null `contentDescription` semantics property.



Step 3–5: Implement, run, pass.



Step 6: Commit.

Task 1.3 — Intentional-stub audit (core/testing repos)



Step 1: For each Test*Repository, grep production tests for calls to its TODO("Not yet implemented") methods.



Step 2: For any method actually invoked by a test, implement behaviorally-equivalent logic (duplicate rejection, default seeding, etc. per Third Law). Document any deliberately-unimplemented method with a // unused-by-design: note.



Step 3: Run the dependent tests → PASS. Commit.

Task 1.4 — :proxy verify-gone + scrub stale wording

Operator decision: delete entirely. Recon shows it was already removed 2026-05-06 — nothing to delete.



Step 1: Confirm no live references: `git grep -nE ':proxy|proxy/|buildFatJar|--legacy'` returns only the documenting comments + this plan. Grep docs/ for any prose still describing :proxy as a usable fallback.



Step 2: Scrub any doc that still presents :proxy/--legacy as a current option (README, ARCHITECTURE, AGENTS) → state it was removed in SP-2. No code change.



Step 3: Commit docs(proxy): correct stale legacy-fallback wording – :proxy removed 2026-05-06.

Constitution gates: §6.J falsifiability per test; §6.AB completeness (rendering + state-machine + gating assertions); §6.AC telemetry on new error paths; §6.S CONTINUATION update.

Exit criteria: Zero // TODO-unwired side-effects in product code; a11y descriptions present; stub ledger written; :proxy decision doc committed.

Phase 2 — Security & quality scanning infrastructure (*containerized, non-interactive*)

Objective: Stand up Snyk + SonarQube + Detekt + golangci-lint + Go-vet + coverage tooling, all via rootless Podman/Compose; wire into scripts/ci.sh; scan; triage; fix every finding.

Task 2.1 — SonarQube + Scanner as Compose services

Files: Create docker-compose.sonar.yml (SonarQube CE + Postgres for Sonar, network bound to 127.0.0.1, rootless); sonar-project.properties; scripts/sonar-scan.sh (thin glue, no sudo); docs/scripts/sonar-scan.sh.md.



Bring up SonarQube via `podman compose -f docker-compose.sonar.yml up -d` (rootless; no privileged). Wait for health.



Configure Kotlin + Go analysis (`sonar.sources`, `sonar.kotlin.*`, `sonar.go.coverage.reportPaths`, exclusions for submodules/, releases/, generated code).



Run scanner container; export report to `.lava-ci-evidence/completeness-program/sonar/<date>/`.

Task 2.2 — Snyk via container

Files: Create scripts/snyk-scan.sh (runs snyk/snyk container image with `snyk test/snyk code test` against the workspace; token from `.env` `SNYK_TOKEN`, §6.R; no interactive auth — `snyk auth` is interactive and is **forbidden**, use `SNYK_TOKEN` env), docs/scripts/snyk-scan.sh.md, .snyk policy file.



Run dependency + code scan (Gradle + Go modules) inside the container; export SARIF/JSON to evidence dir.

Task 2.3 — Detekt + golangci-lint + go vet gates

Files: Create `config/detekt/detekt.yml` + apply detekt convention in `buildSrc`; add `golangci-lint` container invocation in `lava-api-go/scripts/lint.sh`; wire all into `scripts/ci.sh --full`.

Task 2.4 — Coverage tooling

Files: Add **Kover** (Kotlin coverage) convention plugin in `buildSrc`; add `aggregate go test -coverprofile rollup` in `lava-api-go/Makefile`; emit machine-readable reports for Sonar ingestion + the §6.D ledger.

Task 2.5 — Triage + fix loop



For every Snky/Sonar/Detekt/vet finding: classify (real vs false-positive), fix real ones TDD-first with a regression test, and for false-positives add a suppression with a cited reason. Record the full triage in `docs/security/2026-06-04-scan-triage.md`. **No finding left unaddressed.**

Constitution gates: §6.U (no `sudo` — all containerized), §6.R (tokens from `.env`), §6.A real-binary contract test for each new script, §6.J falsifiability for each gate.

Exit criteria: `scripts/ci.sh --full` runs Snky+Sonar+Detekt+lint+vet; zero unaddressed findings; coverage reports generated; triage doc committed.

Phase 3 — Concurrency hardening + leak/deadlock/race sweep + responsiveness

Objective: Prove the no-hazard posture with reproducing tests; consolidate onto `Submodules/{Concurrency,RateLimiter}` primitives; add lazy loading/init, semaphores, non-blocking paths everywhere a blocking spot is found; guarantee flawless responsiveness.

Task 3.1 — Systematic hazard sweep with tests

Files: `core/*/src/test/.../concurrency/*Test.kt`, `lava-api-go/internal/**/race_test.go`, evidence under `.lava-ci-evidence/completeness-program/concurrency/`.



Run Go race detector across the suite: `go test -race ./...` (record + fix any finding).



Audit Android for: `NsdManager/BroadcastReceiver/lifecycle-observer` registration without symmetric removal (leaks); unconfined `launch`; main-thread blocking; shared mutable `var`. For each risk, write a reproducing test (LeakCanary-style assertion or coroutine-test race) then fix.



Audit Go for: goroutines without context cancellation, unguarded shared maps, lock-ordering deadlock potential. Reproduce → fix.

Task 3.2 — Lazy loading / lazy init / semaphores / non-blocking



Introduce by `lazy / sync.Once` for expensive singletons identified by metrics (Phase 5 feeds this); add semaphore-bounded concurrency where unbounded fan-out exists; convert blocking calls on hot paths to `withContext(Dispatchers.IO) / async` non-blocking. Each change ships with a before/after micro-benchmark.

Task 3.3 — Consolidate onto submodule primitives (Decoupled Reusable Architecture)



Where inline `circuit-breaker/semaphore/limiter` duplicates submodules/`concurrency` or `RateLimiter`, migrate to the submodule primitive (or document why the inline version is Lava-domain-specific). Tests stay green throughout.

Constitution gates: §6.T.2 resource limits on test runs; §6.J falsifiability; no-break guarantee.

Exit criteria: go test -race clean; documented leak/deadlock/race audit with reproducing-then-fixed tests; responsiveness micro-benchmarks recorded; primitive-consolidation decisions logged.

Phase 4 — Tests to behavioral maximum (ALL types, every module) (*sub-plan:* *2026-06-...-phase4-coverage.md*)

Objective: Every public surface of every core/*, feature/*, lava-api-go package, and :api-app has real-stack behavioral tests; coverage to meaningful 100% with a ledger for exemptions; one Challenge per feature module (§6.AE backfill).

Workstreams (each a sub-plan section):

- ☐ Per-module unit + orbit ViewModel tests (real UseCase/Repository per Second/Third Law).
- ☐ Integration Challenge Tests per feature (close §6.AE-debt; flip check -challenge-coverage.sh to STRICT).
- ☐ Go contract/parity/e2e expansion to every handler + the new scan-driven cases.
- ☐ Go **fuzz** tests for parsers (rutracker HTML, DTO decode).
- ☐ Screenshot/visual-diff tests for key Compose screens (catches the §6.AB white-icon class).
- ☐ **Mutation testing** wired as a gate (Pitest for Kotlin where feasible / Go mutation tooling) to prove tests kill mutants.
- ☐ Coverage ledger docs/coverage/<module>-exemptions.md for every uncovered line.

Exit criteria [GATE-HOST for Challenges]: Aggregate coverage report at target; check-challenge-coverage.sh STRICT-green; mutation score recorded; ledger complete.

Phase 5 — Stress / load / chaos / metrics-driven optimization (*sub-plan*)

Objective: Prove the system is "responsive like the flash and not possible to overload or break"; collect metrics; optimize from data.



Load tests against lava-api-go (k6/vegeta container, rootless) — p50/p95/p99 latency + throughput SLOs; assert non-degradation under concurrency.



Stress tests — push to saturation, assert graceful degradation (rate-limiter + circuit-breaker engage, no crash, no deadlock).



Chaos tests (§11.4.85) — kill dependencies (Postgres, upstream tracker), assert recovery + non-fatal telemetry fires.



Metrics/monitoring-assertion tests — assert Prometheus counters/histograms move correctly under load (so dashboards aren't bluffs).



Optimization loop — feed Phase-3 lazy-init/semaphore/non-blocking work from these numbers; re-measure; record deltas.

Exit criteria: SLO doc with measured p99 + saturation behavior; chaos-recovery evidence; metrics-assertion tests green; optimization before/after deltas committed.

Phase 6 — Documentation to nano-detail (*sub-plan*)



REST API reference for lava-api-go (from api/openapi.yaml → rendered docs/api/).



Deployment guide (lava-api-go OCI + compose, on-device API, client signing/distribute).



Security hardening doc (auth/HMAC, credential handling §6.H, TLS, scan results posture).



Database ERD + docs/db/schema.md (Room 11.json + Postgres 0001–0009, generated diagram).



Client user manual + install/setup + per-feature tutorials (search, browse, topic, download, forum, onboarding, on-device API) + troubleshooting.



Refresh ARCHITECTURE.md, sdk-developer-guide.md, README.md, AGENTS.md to current 1.3.x/0.2.x.



Per §11.4.65, regenerate .html/.pdf exports for every new doc (scripts/sync-markdown-exports.sh).

Exit criteria: All Section-D gaps closed; export pipeline green.

Phase 7 — Video courses (*sub-plan*)



Author docs/courses/lava-client/ + docs/courses/lava-api/ lesson scripts + outlines (mirroring submodule docs/courses/lesson-*.md structure) covering install→search→download→forum→on-device API→troubleshooting, and a developer course (SDK plugin recipe, architecture, build/test/distribute).



Storyboards + slide outlines + recordable narration scripts; asset list. (Actual recording is an operator step; the repo carries the complete script/storyboard/slides.)

Exit criteria: Complete, current course scripts + storyboards committed; cross-linked from README/website.

Phase 8 — Website (*sub-plan*)



Stand up an in-repo docs/marketing site (website/, docusaurus or mkdocs-material) building **the Phase-6 docs + Phase-7 course outlines**; CI builds it locally (Local-Only CI/CD rule — no hosted pipeline).



Content current to 1.3.x client + 0.2.x api-app + 2.3.x API; download/links point to Firebase (no dead Play-Store links per the linking feature).



Build verification in `scripts/ci.sh --full` (containerized node build, rootless).

Exit criteria: website/ builds locally; content current; linked from README.

Phase 9 — Diagrams + SQL definitions (*sub-plan*)



Add `.mmd` diagram assets to `docs/` (architecture, module graph, mDNS discovery sequence, on-device API JNI flow, auth/HMAC sequence, request lifecycle) — render to SVG/PNG for the website.



Unified `docs/db/schema.md` + ERD; document every migration's intent; cross-link Room↔Postgres parity where relevant.

Exit criteria: Diagram assets present + rendered; SQL fully documented.

Phase 10 — Constitutional debt closure



\$6.Y-debt + \$6.Z-debt + \$6.AA-debt + \$6.AB-debt + \$6.AC-debt — implement the owed pre-push/lint/gate mechanics + hermetic tests (each closes per its -debt spec in CLAUDE.md).



\$6.AD-debt remaining (CM-SCRIPT-DOCS-SYNC per-script, CM-BUILD-RESOURCE-STATS-TRACKER).



\$6.AH-debt — no-KVM/TCG containerized emulator boot (Containers submodule) **or** provision a Linux x86_64+KVM gate-host; either unblocks the device gates this plan depends on.



\$11.4.109 UNCONFIRMED: → operator verbatim quote; docs_chain submodule incorporation.

Exit criteria: All -debt items CLOSED or explicitly RESOLVED-by-equivalence with ledger; verify-all sweep STRICT-green.

Phase 11 — Final verification + distribute



[GATE-HOST] Full `scripts/ci.sh --full` + \$6.AE Challenge matrix (min API 28/30/34/latest × phone/tablet) green.



\$6.Y bump → \$6.P CHANGELOG → \$6.Z evidence → \$6.AA two-stage Firebase distribute (debug → verify → release) for client + api-app; lava-api-go image to registry.



Resolve CONTINUATION cloud-search lag (`scripts/distribute-api-remote.sh` when `thinker.local` reachable).



Final CONTINUATION.md + memory handoff; converge GitHub+GitLab.

Exit criteria: Everything green on the gate-host; both apps distributed both stages; CONTINUATION current; mirrors converged.

Self-Review (writing-plans checklist)

1. **Spec coverage:** Every clause of the request maps to a phase — unfinished report (Part I), dead-code (P1), all test types + Challenges (P4), 100% coverage + ledger (P4), docs/manuals/courses/website/diagrams/SQL (P6–9), leaks/deadlocks/races (P3), Snyk+SonarQube containerized (P2), metrics→optimization (P5), lazy/semaphore/non-blocking (P3+P5), stress/integration (P4+P5), no-break (P0 baseline + per-task green), constitution respect (gates each phase), no interactive root (P2 design + \$6.U).
2. **Placeholder scan:** Phases 0–3, 10, 11 carry concrete bite-sized tasks; Phases 4–9 are explicitly deferred to per-phase sub-plans (skill-sanctioned for multi-subsystem scope) — not hand-waved, each with files + exit criteria.

3. **Type/name consistency:** Side-effect names (ShowAddCommentDialog/ShowAddCommentError) verified against TopicScreen.kt; script names match docs/scripts/*.md convention.

Execution Handoff

This master plan spawns sub-plans for Phases 4–9 at their start. Phases 0–3 are ready to execute now.